Containerization

The Next Generation Virtualization for Cloud

Prof. B. Thangaraju, IIITB.





Agenda

- Docker Theory
- Docker Lab

Agenda –Docker Theory

- Introduction to Virtualization
- Virtual Machine Vs Container
- Docker Internals
- Docker for Lab
- Spectrum of Cloud Computing Services
  - IaaS, CaaS, PaaS, SaaS, …
- Most Popular CaaS Platforms
- Recent Trends and Future Prospects
- Conclusion

Introduction to Virtualization

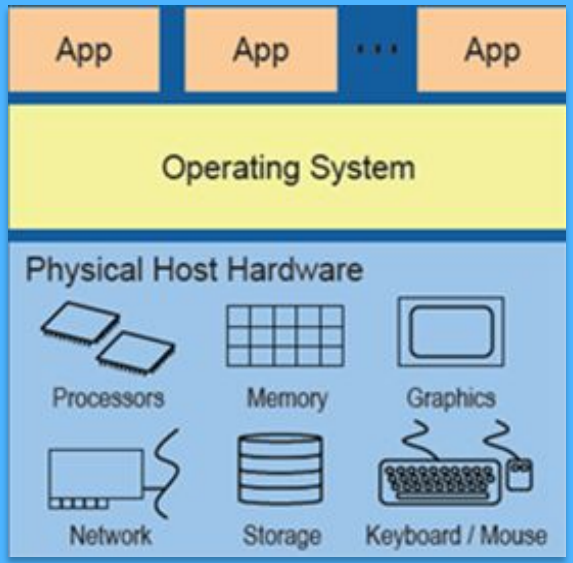
- **Software demands more** and more from operating systems to applications.
  - More : data, processing power, memory, network bandwidth and storage space.
- **Virtualization is a technology**.
  - Create multiple simulated environments from a single, physical hardware systems.
- **Software called a hypervisor** connects directly to the hardware.
  - Split one system into separate, distinct, and secure environments known as virtual machines (VMs).

Introduction to Virtualization

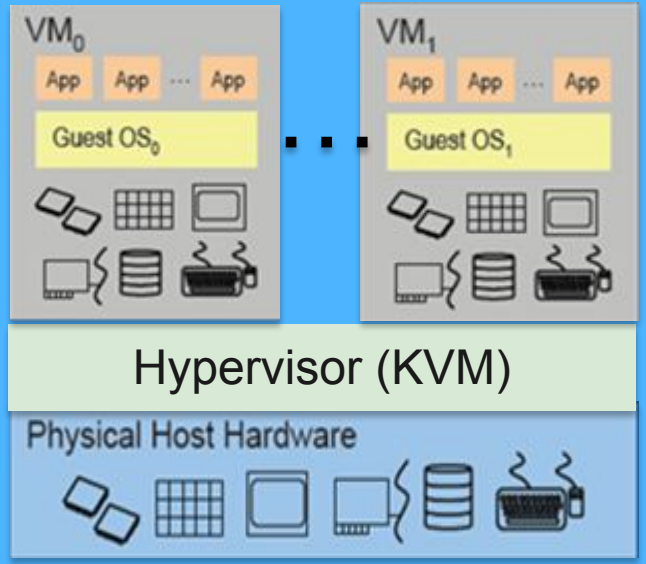
The term **hypervisor** is a variant of supervisor, a traditional term for the kernel of an operating system: the *hypervisor is the* ***supervisor of the supervisors***

- The hypervisor/**VMM is the control system** at the core of virtualization.
  - Acts as the control and translation system between the VMs and the hardware.
- **Hypervisors need OS** - level components to run VMs
  - Memory manager, Process scheduler, I/O stack, DD, security manager and Network stack.
- The physical hardware, equipped with a hypervisor is called the **Host**, while the many VMs that use its resources are **Guests**.

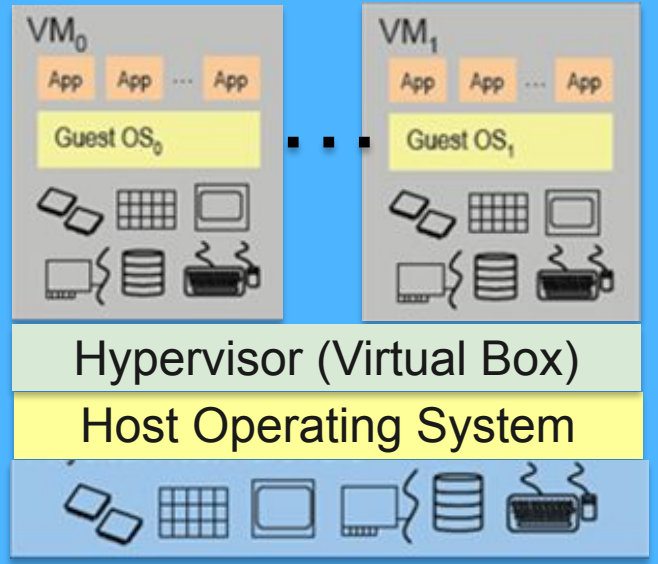
Types of Virtualization



Physical System



Type 1 Hypervisor
Native/Bare-Metal Architecture



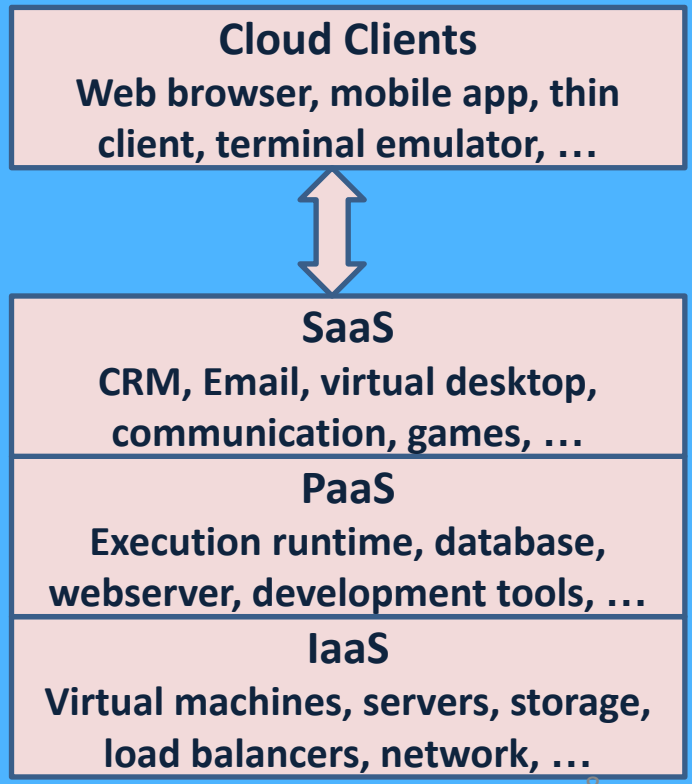
Type 2 Hypervisor
Hosted Architecture

Benefits of Virtualization

- Virtualization is being used by a growing number of organizations
  - Reduce power consumption and air conditioning needs (Move to be more Green-Friendly)
  - Trim the building space and land requirements
  - Reduce downtime and provides high availability for critical applications (Blue-Green model)
  - Simplify IT operations
  - Allow IT organizations to respond faster to changing business demands

Cloud Computing

- Cloud computing is a set of principles and approaches
  - To deliver compute, network, and storage infrastructure resources, services, platforms, and applications to users on-demand across any network.
  - These infrastructure resources, services, and applications are sourced from clouds, which are pools of virtual resources orchestrated by management and automation software.
  - Resources can be accessed by users on-demand through self-service portals supported by automatic scaling and dynamic resource allocation.



Virtualization Vs Cloud Computing



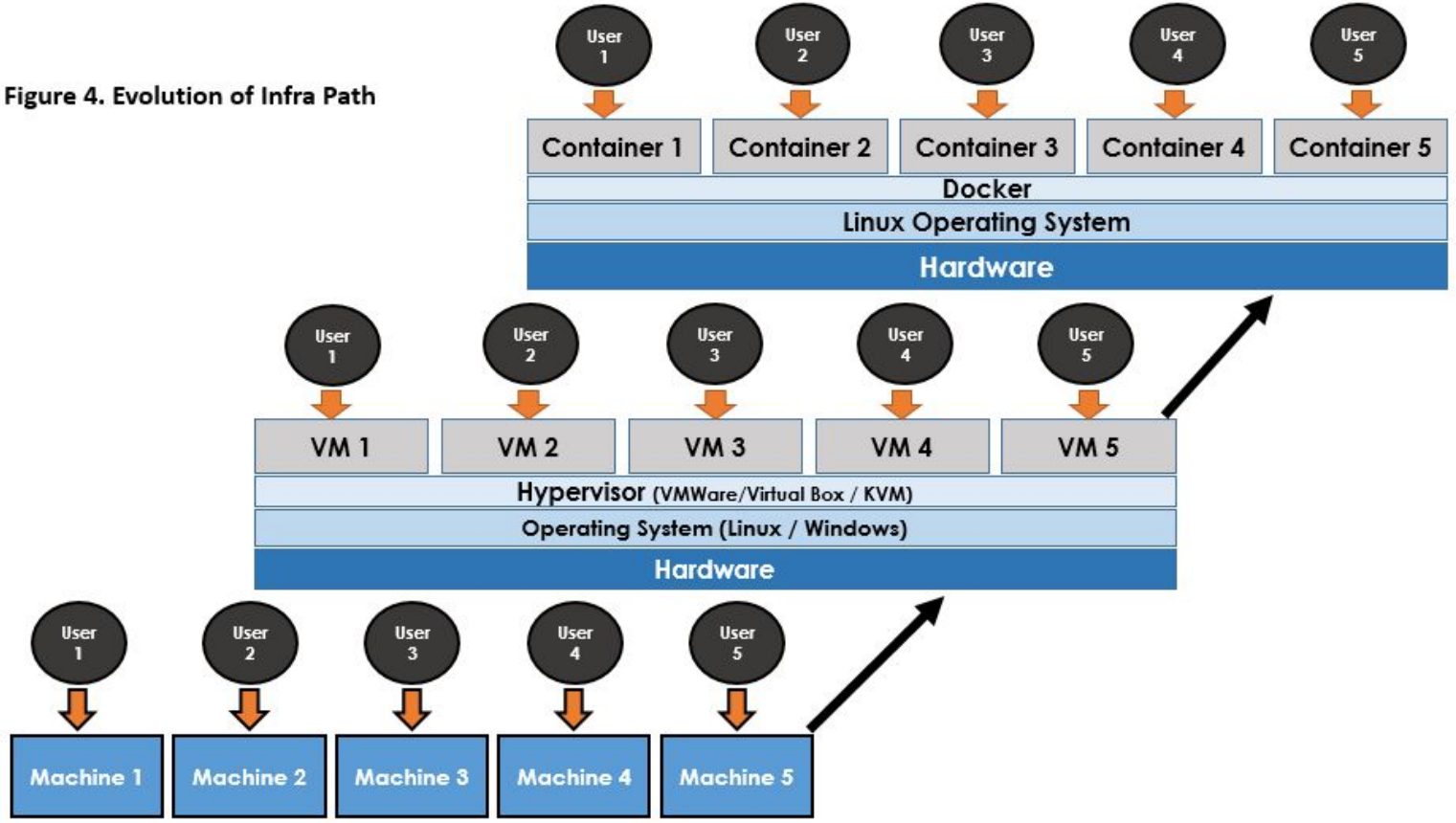


	Virtualization	Cloud
Definition	Technology	Methodology/Environment
Purpose	Create multiple simulated environments from 1 physical hardware system	Pool and automate virtual resources for on-demand use
Configuration	Image-based	Template-based
Cost	High capital expenditures (CAPEX), low operating expenses (OPEX)	Private cloud: High CAPEX, low OPEX Public cloud: Low CAPEX, high OPEX
Other Benefits	❑ Effective Server Utilization ❑ Easy Server Maintenance ❑ Separated Physical Infrastructure ❑ Infrastructure Cost Savings	❑ Automated Management ❑ Scalability ❑ Self-serving ❑ Pay-As-You-Go

Interdependent but Not Interchangeable

Evolution of Infra Path

Figure 4. Evolution of Infra Path



Containerization

- **Containerization is a Modern Virtualization method**
  - defined as a form of **operating system virtualization**.
  - accesses a single OS kernel to power multiple distributed applications that are each developed and run in their own container.
  - Application with all dependencies and libraries needed are packed as an image that can be run anywhere - **desktop, traditional IT, or the cloud**.

History of Container Technology

- Chroot circa 1982
- FreeBSD Jails circa 2000
- Solaris Zones circa 2004
- Meiosys - MetaClusters with Checkpoint/Restore 2004-05
- Linux OpenVZ circa 2005 (not in mainstream Linux)
- AIX WPARs circa 2007
- LXC circa 2008
- Systemd-nspawn circa 2010-2013
- Docker circa 2013
  - built on **LXC**
  - moved to **libcontainer** (March 2014)
  - **appC** (CoreOS) announced (December 2014)
  - Open Containers standard for **convergence** with Docker Announced (June 2015)
  - moved to **runC** (OCF compliant) (July 2015)
- … many more container formats coming?

Docker –An Introduction

- **Docker (Dictionary meaning) – longshoreman**
  - somebody whose job is to load and unload cargo vessels in a port.
- **Docker (OSS meaning)**
  - Automates the deployment of applications inside software containers.
- Licensed under the Apache 2 .
- **Docker CE**: The core Docker platform, which Docker calls ***Docker Community Edition***, is available for anyone to download and run free of charge.
- **Docker EE**: Docker offers three **enterprise editions** of its software. Pricing starts at $750 per node per year. –Ref: https://www.docker.com/pricing
- Docker source code is available at: https://github.com/docker/docker.

Docker –An Introduction

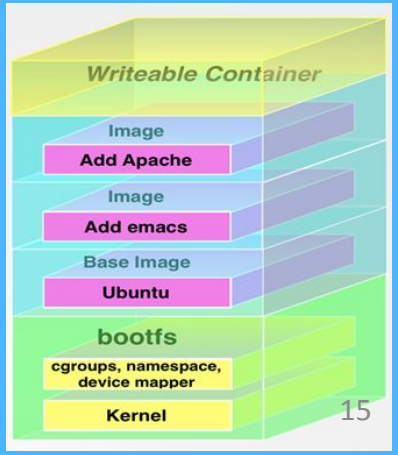
- Docker can be run on any x64 host running a modern Linux kernel version 3.8 and later. Currently x86_64 and amd64 only and 32-bit is **NOT** supported. It includes:
- Docker implements a high level API to provide lightweight containers that run processes in isolation.
- Isolation features
  - namespaces and cgroup to allow independent containers to run avoiding the overhead of starting virtual machines.
- By using containers, resources can be isolated, services restricted and processes

Docker –An Introduction

- Docker is an open source tool that helps in the management of containers.
- It was launched in 2013 by a company called dotCloud. Only a few months later, dotCloud, Inc. changed its name to Docker, Inc. as the Docker project had became so popular.
- It is written in Go and uses Linux kernel features like namespaces and cgroups.
- **Use Cases**: Application development, testing, packaging and deployment. PaaS, SaaS and Cloud Infrastructure. Application and service isolation.

Docker Internals

- Docker makes use of Linux kernel **namespaces** to provide the isolated workspace called the *container*.
- When you run a container, Docker creates a set of *namespaces* for that container. These namespaces provide a layer of isolation.
- Each aspect of a container runs in a separate namespace and its access is limited to that namespace.
- Docker Engine uses the following namespaces on Linux:
  - **PID namespace** for process isolation.
  - **NET namespace** for managing network interfaces.
  - **IPC namespace** for managing access to IPC resources.
  - **MNT namespace** for managing filesystem mount points.
  - **UTS namespace** for isolating kernel and version identifiers.

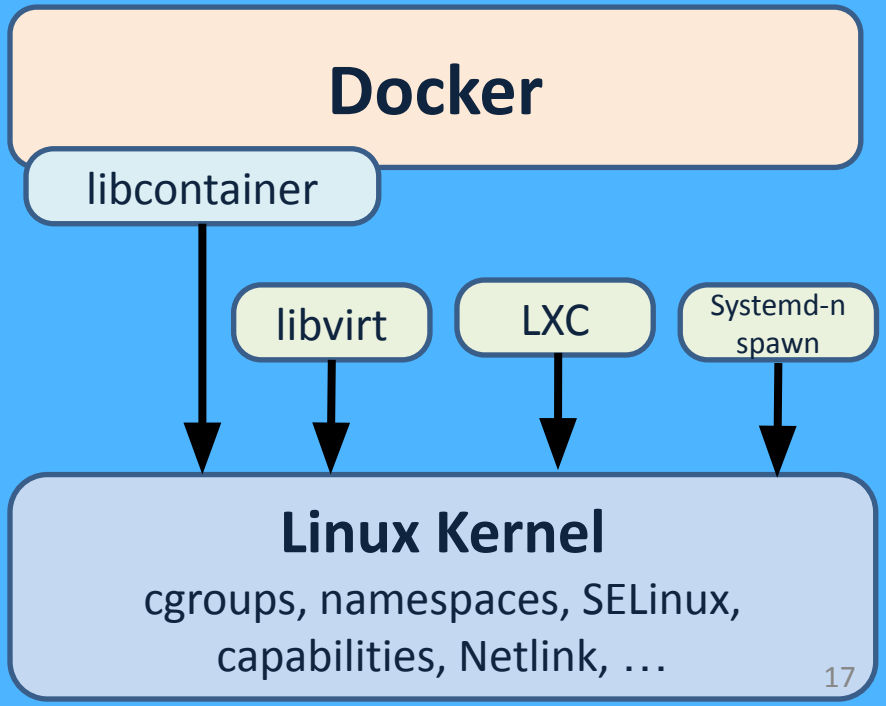


Docker Internals

- Control groups allow Docker Engine to share available hardware resources to containers and optionally enforce limits and constraints.
- Docker Engine uses the following cgroups:
  - **Memory cgroup** for managing accounting, limits and notifications.
  - **HugeTLB cgroup** for accounting usage of huge pages by process group.
  - **CPU group** for managing user / system CPU time and usage.
  - **CPUSet cgroup** for binding a group to specific CPU. Useful for real time applications and NUMA systems with localized memory per CPU.
  - **BlkIO cgroup** for measuring & limiting amount of blckIO by group.
  - **net_cls** and **net_prio cgroup** for tagging the traffic control.
  - **Devices cgroup** for reading / writing access devices.

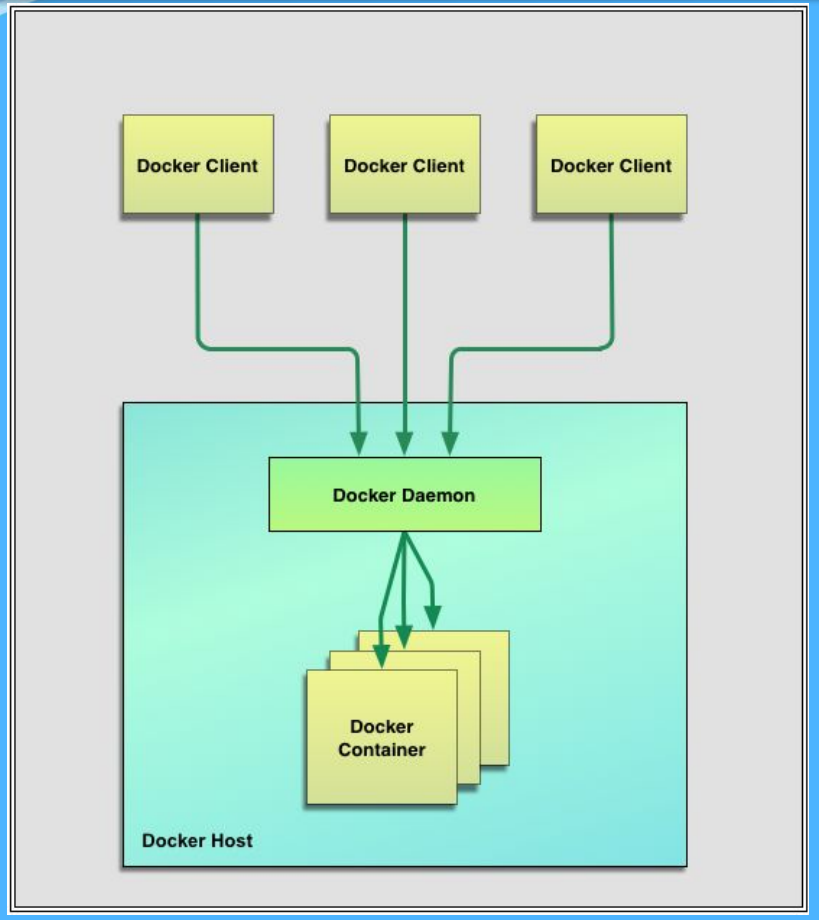
Docker Internals

- **Docker Engine** combines the namespaces, control groups, etc., into a wrapper called a container format.
- The default container format is libcontainer.
- ***libcontainer*** library as a reference implementation for containers, and build on top of libvirt, LXC and systemd-nspawn.
- **libvirt** is an open-source API, daemon and management tool for managing platform virtualization.
- **LXC** -Combines kernel's cgroup and namespaces to provide an isolated environment.
- **Systemd-nspawn** - used to run a command in light weight namesapce container.

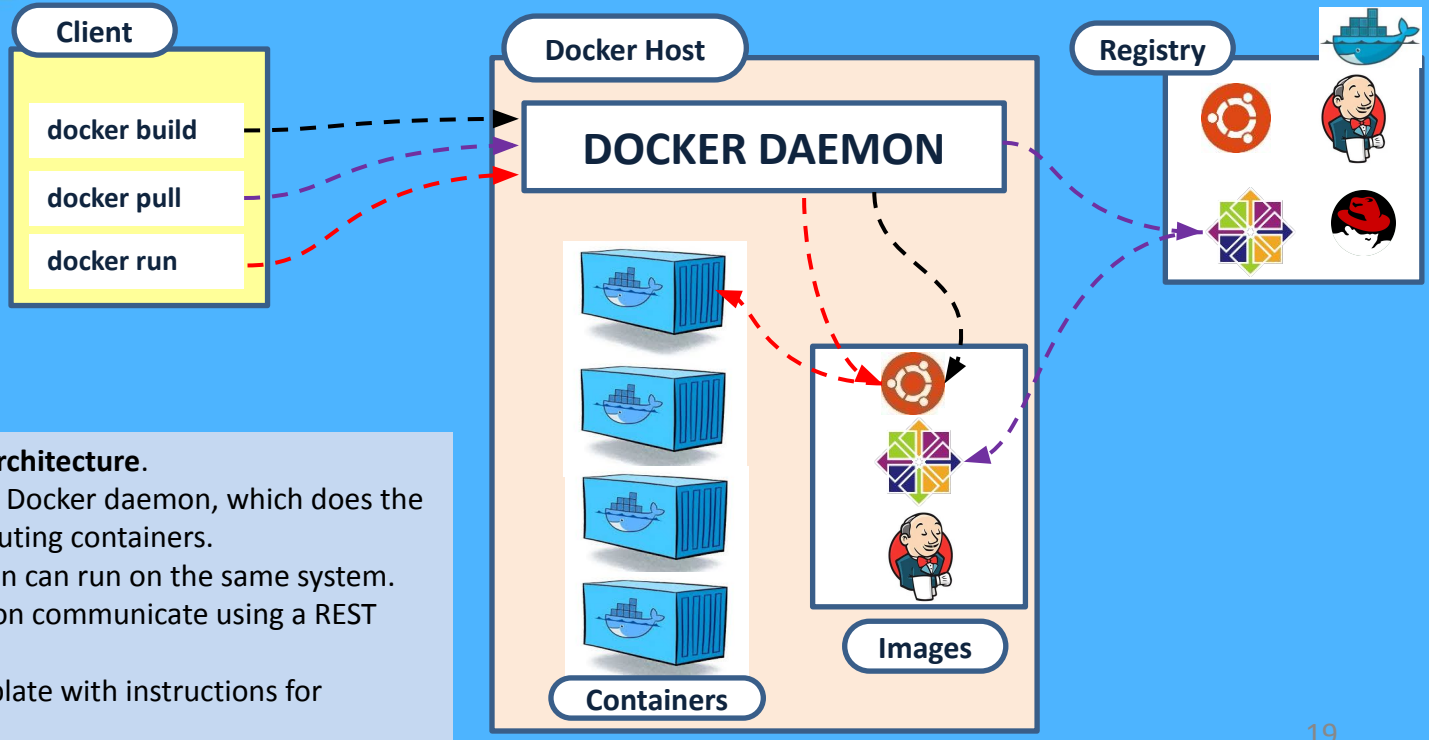


Docker Architecture

- The core components that compose Docker:
  - The Docker client and server
  - Docker Images
  - Registries
  - Docker Containers
- Docker is a client-server application. The Docker client talks to the Docker server or daemon, which, in turn, does all the work.



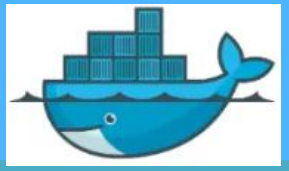
Docker Architecture



- Docker uses a **client-server architecture**.
- The Docker client talks to the Docker daemon, which does the building, running, and distributing containers.
- The Docker client and daemon can run on the same system.
- The Docker client and daemon communicate using a REST API, over UNIX sockets.
- An ***image*** is a read-only template with instructions for creating a Docker container.

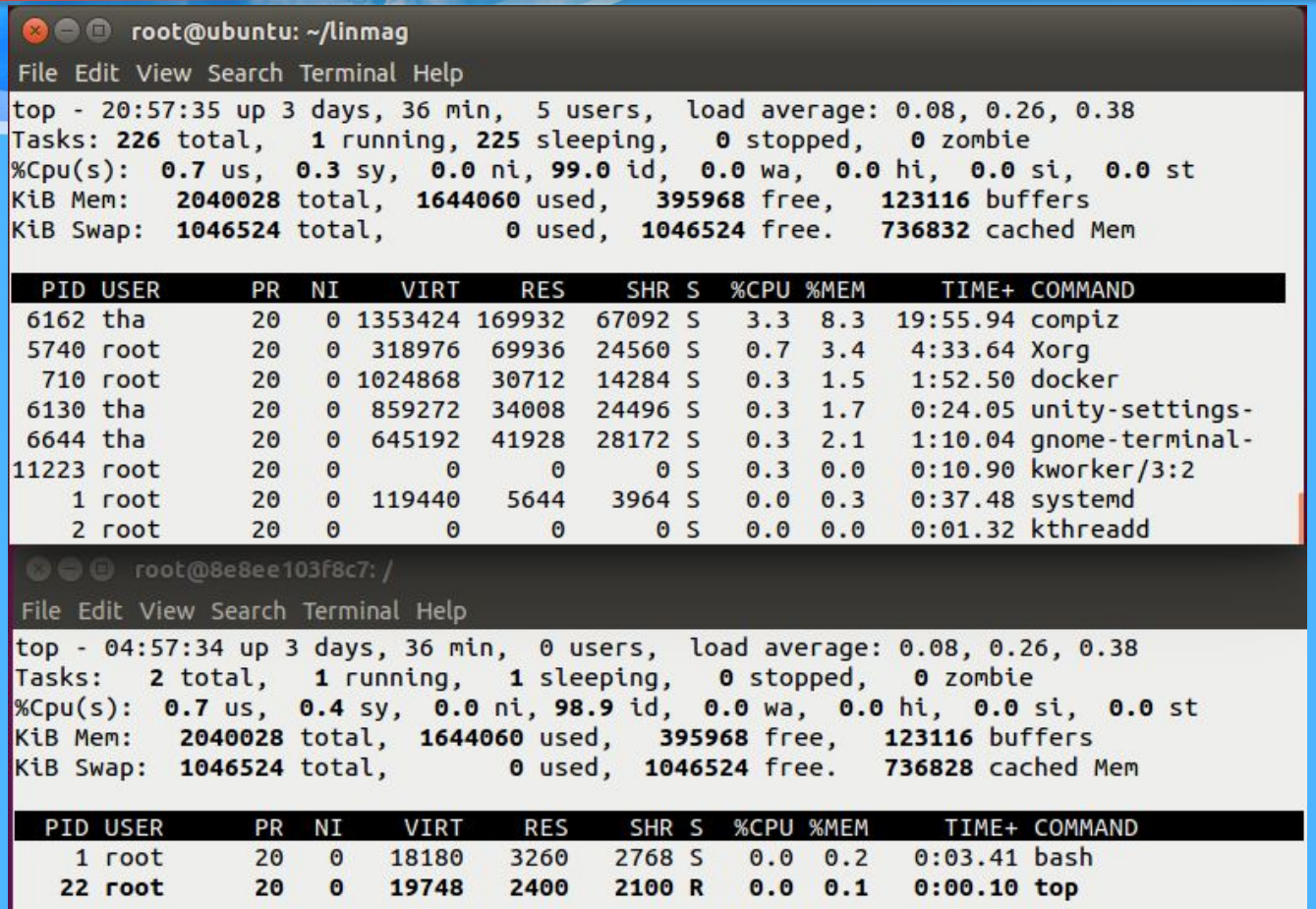
Virtualization Vs Container



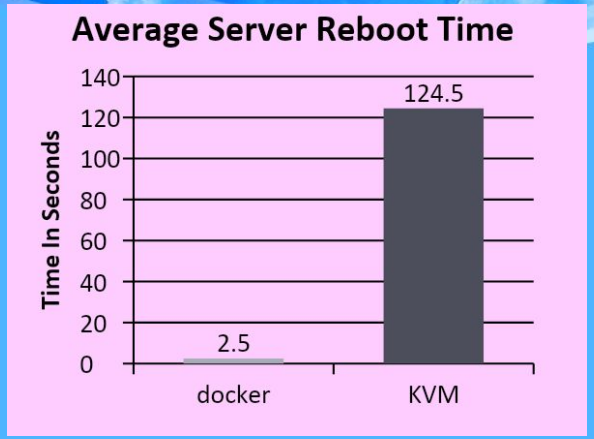


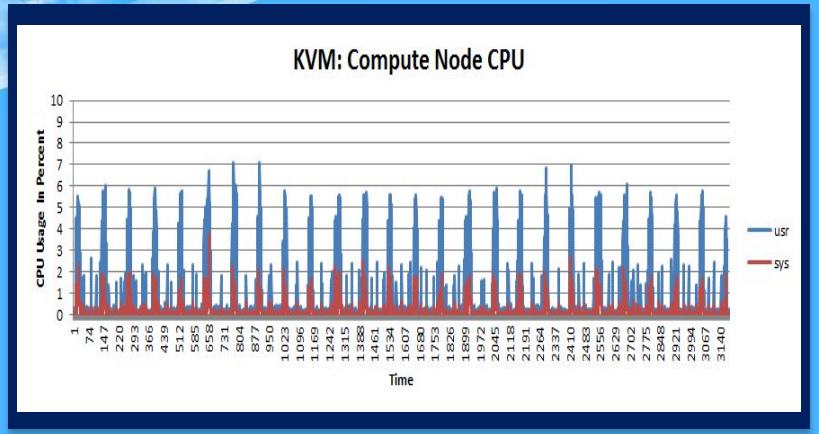
Virtualization	Container
Hardware Level Virtualization	OS Level Virtualization
Typically measured by the gigabyte	Typically measured by the megabyte
Traditional IT architectures	**Emerging IT practices**
House traditional, legacy, and monolithic workloads	cloud-native development, CI/CD, and DevOps
All the code and dependencies in one place led to oversized VMs that experienced cascading failures and downtime when pushing updates.	Workloads are broken into the smallest possible serviceable units and packaged in containers.

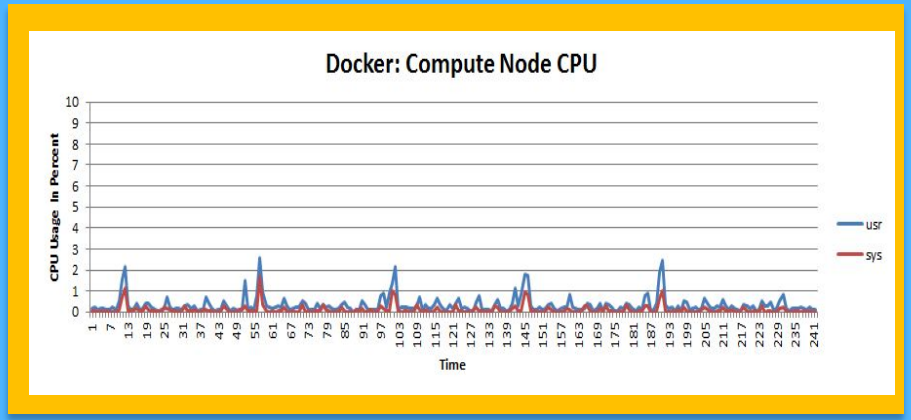
VM Vs Docker


root@ubuntu: ~/linmag
File Edit View Search Terminal Help
top - 20:57:35 up 3 days, 36 min, 5 users, load average: 0.08, 0.26, 0.38
Tasks: 226 total, 1 running, 225 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.7 us, 0.3 sy, 0.0 ni, 99.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
KiB Mem: 2040028 total, 1644060 used, 395968 free, 123116 buffers
KiB Swap: 1046524 total, 0 used, 1046524 free. 736832 cached Mem
PID USER PR NI VIRT RES SHR S %CPU %MEM TIME+ COMMAND
6162 tha 20 0 1353424 169932 67092 S 3.3 8.3 19:55.94 compiz
5740 root 20 0 318976 69936 24560 S 0.7 3.4 4:33.64 Xorg
710 root 20 0 1024868 30712 14284 S 0.3 1.5 1:52.50 docker
6130 tha 20 0 859272 34008 24496 S 0.3 1.7 0:24.05 unity-settings-
6644 tha 20 0 645192 41928 28172 S 0.3 2.1 1:10.04 gnome-terminal-
11223 root 20 0 0 0 0 S 0.3 0.0 0:10.90 kworker/3:2
1 root 20 0 119440 5644 3964 S 0.0 0.3 0:37.48 systemd
2 root 20 0 0 0 0 S 0.0 0.0 0:01.32 kthreadd
root@8e8ee103f8c7: /
File Edit View Search Terminal Help
top - 04:57:34 up 3 days, 36 min, 0 users, load average: 0.08, 0.26, 0.38
Tasks: 2 total, 1 running, 1 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.7 us, 0.4 sy, 0.0 ni, 98.9 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
KiB Mem: 2040028 total, 1644060 used, 395968 free, 123116 buffers
KiB Swap: 1046524 total, 0 used, 1046524 free. 736828 cached Mem
PID USER PR NI VIRT RES SHR S %CPU %MEM TIME+ COMMAND
1 root 20 0 18180 3260 2768 S 0.0 0.2 0:03.41 bash
22 root 20 0 19748 2400 2100 R 0.0 0.1 0:00.10 top

KVM Vs Docker Performance Benchmark







- Container = isolated process(es).
- Lower overhead than VMs – can run a full system with an init or a single service or process

Ref: Passive Benchmarking with docker LXC, KVM & OpenStack by Boden Russell, IBM Global Technology Service, Advanced Cloud Solutions & Innovation.

Benefits of Containerization

- **Portability** - is portable and able to run uniformly and consistently across any platform or cloud.
- **Agility** - Software developers can continue using agile or DevOps tools and processes for rapid application development and enhancement.
- **Speed** - speeding up start-times as there is no operating system to boot.
- **Fault Isolation** - The failure of one container does not affect the continued operation of any other containers.
- **Efficiency** - containers are inherently smaller in capacity than a VM, this drives higher server efficiencies.
- **Ease of Management** - A container orchestration platform automates the installation, scaling, and management of containerized workloads and services.
- **Security** - security permissions can be defined to automatically block unwanted components from entering containers or limit communications with unnecessary resources.

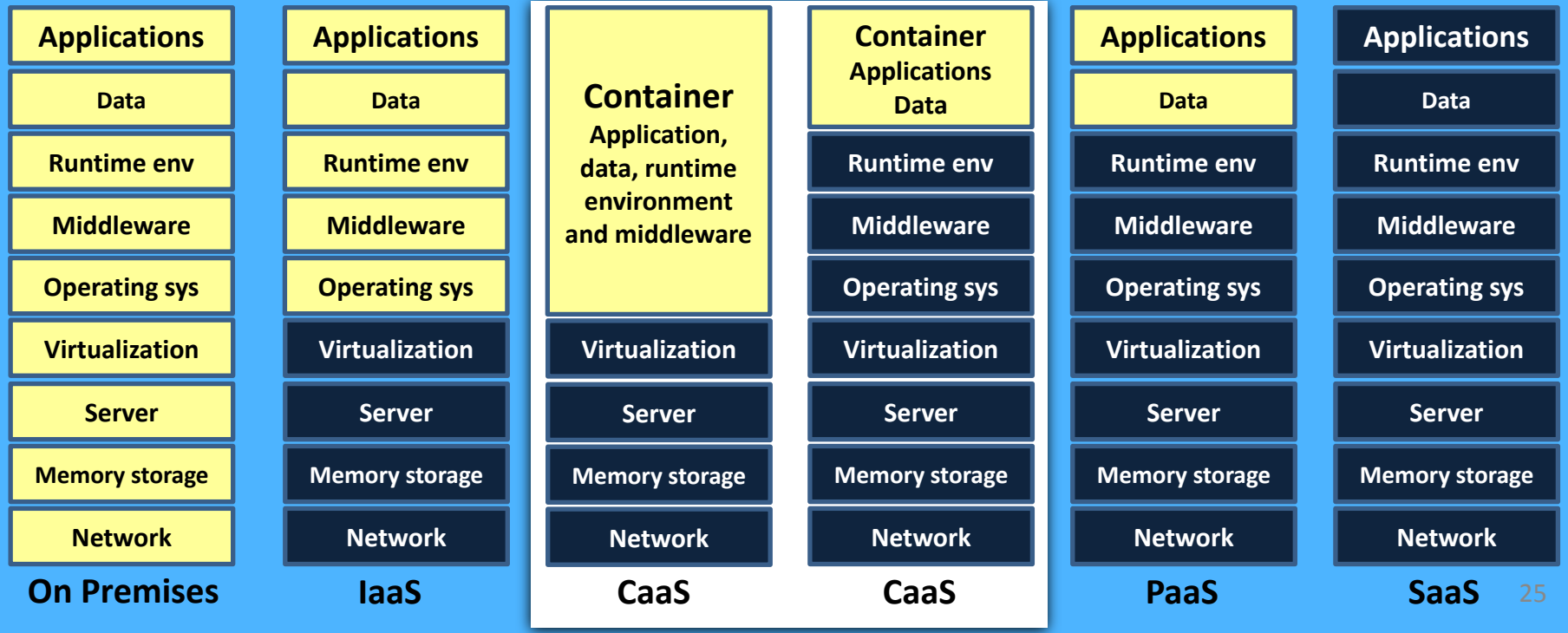


CaaS

- Containerization is one of the latest developments in the evolution of cloud computing, foundation of a private cloud and are becoming a game changer for many organizations.
- Container-as-a-Service is a computer cluster that is available through the cloud and is used by users to upload, create, centrally manage, and run container-based applications on the cloud platform. The interaction with the cloud-based container environment takes place either through the graphical user interface (GUI) or in the form of API calls.
- CaaS is a business model whereby cloud computing service providers offer **container-based virtualization as a scalable online service**.
- CaaS typically refers to a **complete container environment**, including orchestration tools, an image catalog (docker registry), cluster management software, built in functionality for auto scaling and a set of developer tools and APIs.

Spectrum of Cloud Computing Services

- CaaS falls somewhere between Infrastructure as a Service (IaaS) and Platform as a Service (PaaS).
- However, CaaS is most commonly positioned as a subset of IaaS.

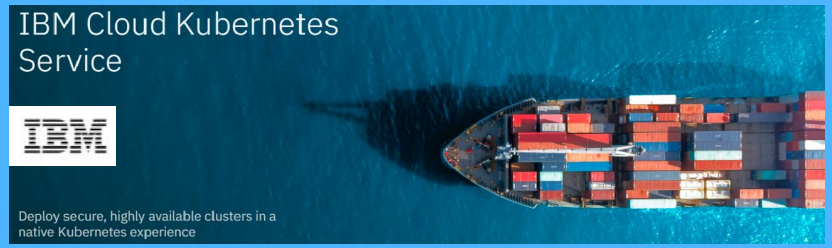


Most Popular CaaS Platforms

- Containers as a Service Market is experiencing significant growth due to demand for the technology.
  - Cost advantages
  - Increasing trend in microservices.
  - Software firms are already in the market
  - Google LLC, Amazon Web Services, Inc, IBM Corporation are funding the advancement of technology.

1. Amazon EC2 Container Service (ECS),
2. Microsoft Azure Container Service (ACS).
3. Google Container Engine (GKE)

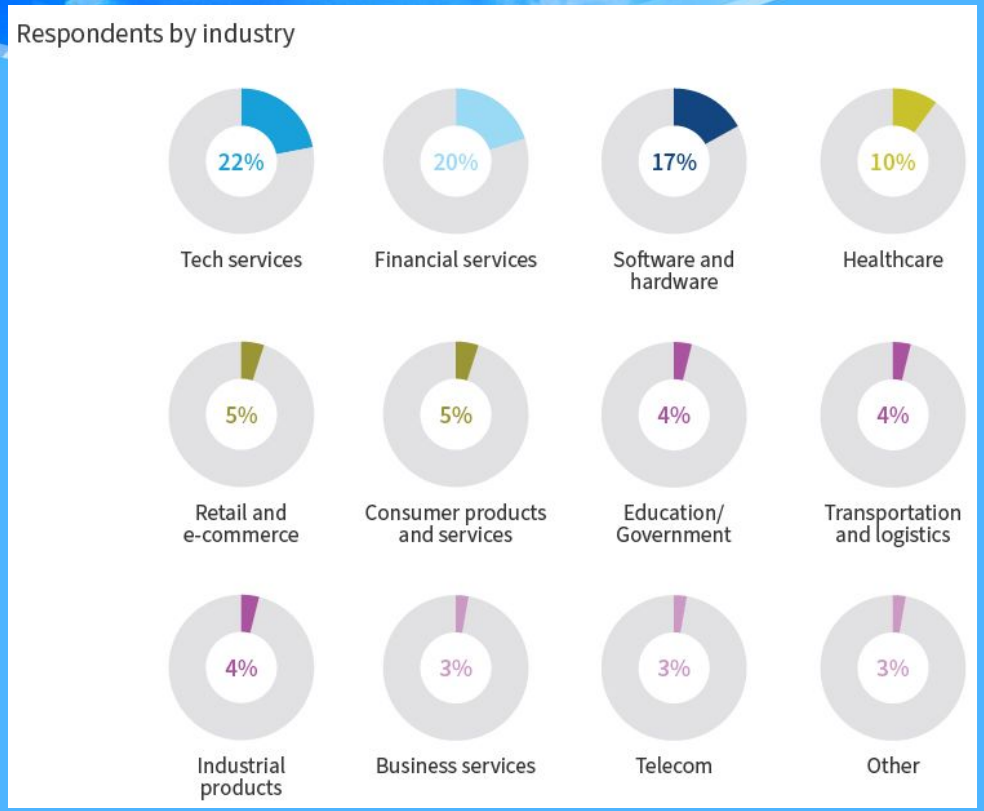






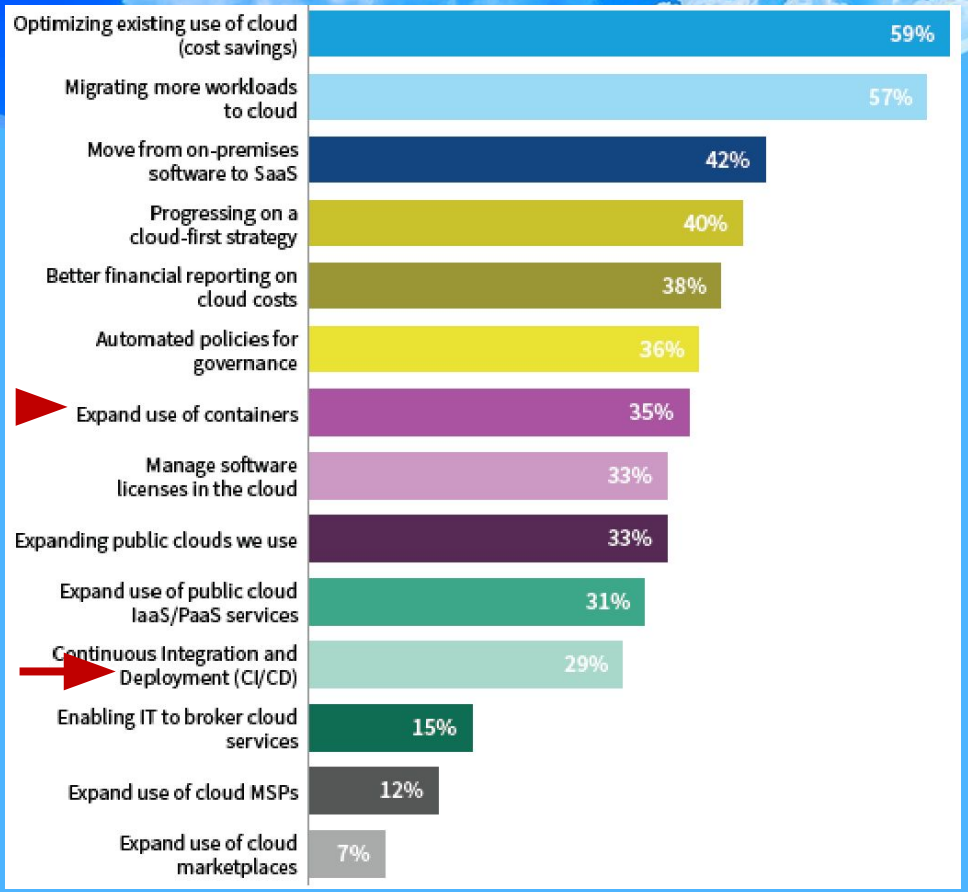


Recent Trends and Future Prospects

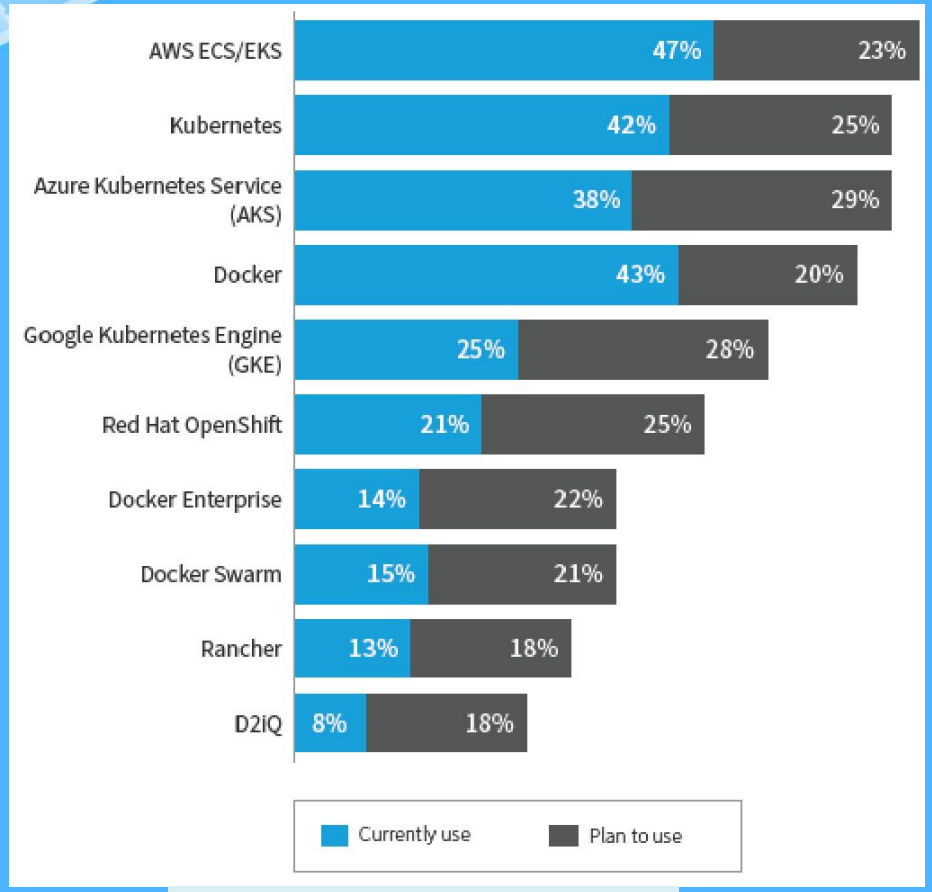




https://resources.flexera.com/web/pdf/Flexera-State-of-the-Cloud-Report-2022.pdf?elqTrackId=f3bb660986704d2980404386aa003141&elqaid=6925&elqat=2



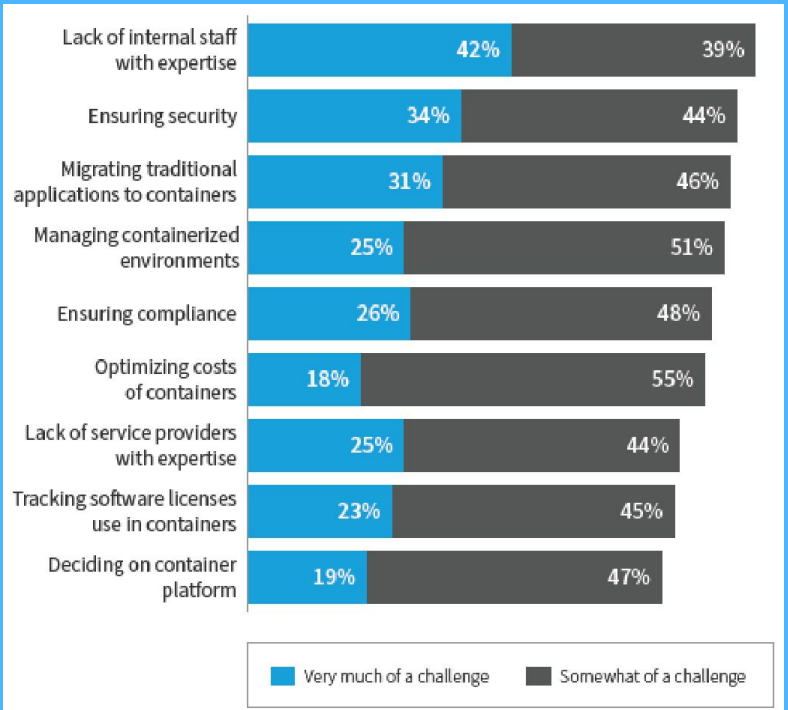
Top cloud initiatives for 2022 across all organizations



Containers Status Report 2022

Containers may be mainstream, but lack of expertise is the top challenge

- Top container challenges for all organizations are the lack of internal resources with expertise, ensuring security and migrating traditional applications to containers.
- Respondents citing lack of internal resources with expertise as a significant challenge grew from 30 percent last year to 42 percent this year.
- The resource challenges can be attributed to the strong uptick in the use of container technology.
- Migrating traditional applications to containers is more problematic, as traditional apps aren't implemented via microservices, which are more aligned to containerized deployment models.

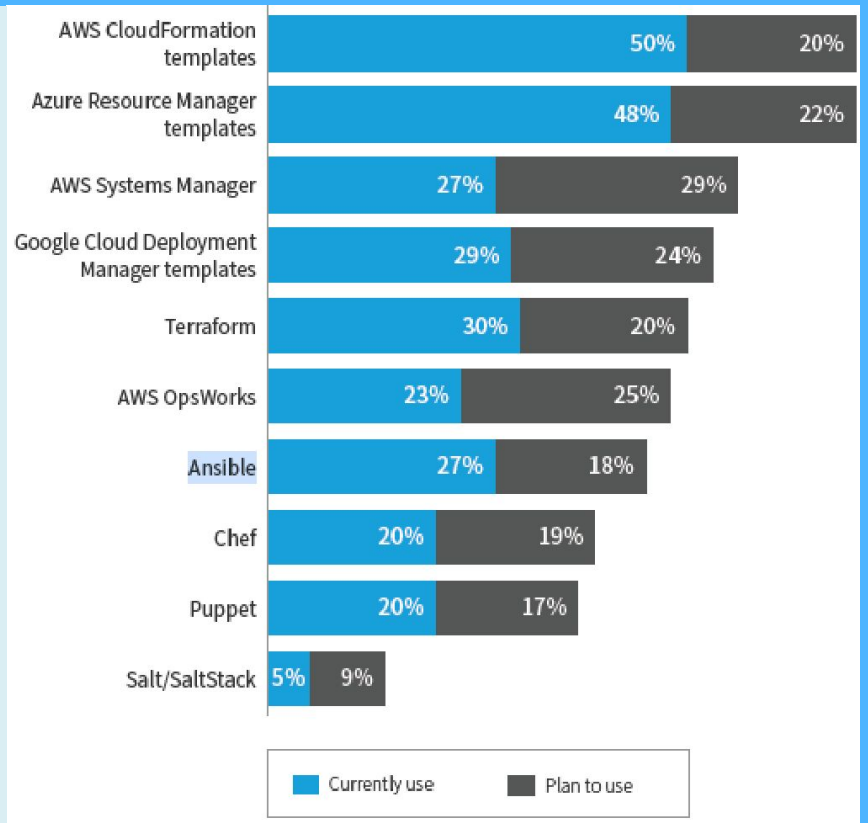


Top container-related challenges for all organizations

Adoption of cloud configuration tools is shifting

- Cloud use often goes hand in hand in hand with adopting DevOps processes.
- Configuration management tools is used to standardize and automate deployment and configuration of servers and applications.
- Native cloud tools are most commonly used today.
- Third-party tool usage declined from last year:
  - Terraform dropped from 36 percent to 30 percent
  - Ansible dropped from 31 percent to 27 percent
  - Chef dropped from 27 percent to 20 percent
  - Puppet dropped from 27 percent to 20 percent
  - Salt/SaltStack dropped from twelve percent to five percent.

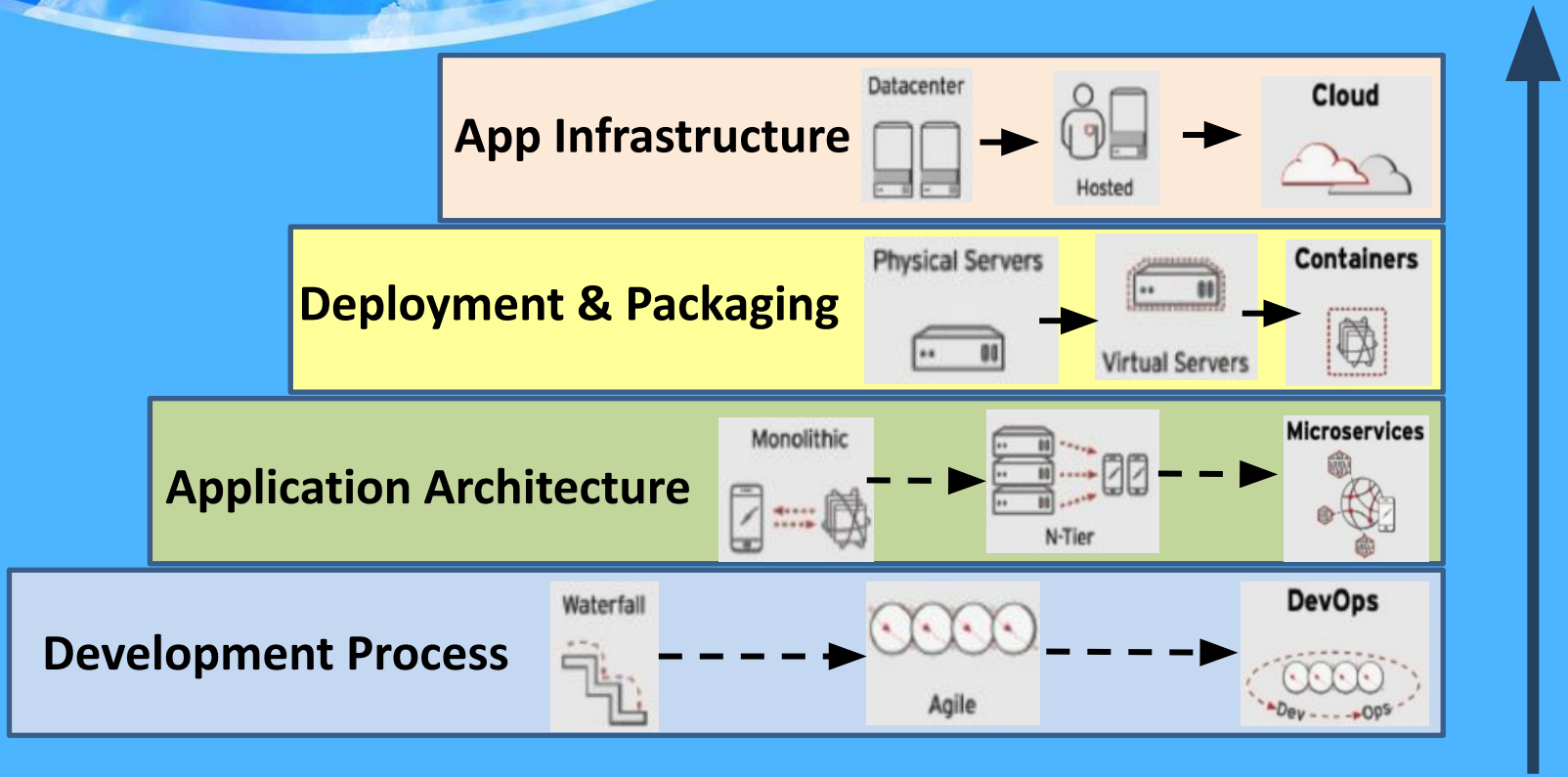
This may indicate that the ease and features of provider-specific tools are more enticing to organizations, despite our multi-cloud world. In comparison, it can be cumbersome to implement a third-party tool and cover all provider services.



Docker Customers



Enterprise Applications - Evolving Themes





Docker Lab

Agenda –Docker Lab

- Basic Docker Commands
- Docker Access Through SSH
- Docker Web Server
- Docker Image Creation
- Docker Volume
- Docker Networking

Docker Basic Commands

Install in Ubuntu: apt-get install docker.io (Ref: https://docs.docker.com/engine/install/ubuntu/)

Check docker service status: service docker status

docker pull ubuntu - **pull docker images from dockerhub**

docker images - **List all the docker images**

Check: docker hub -https://hub.docker.com/

Check image repositories:

ex: $sudo cat /var/lib/docker/image/overlay2/repositories.json | python3 -mjson.tool

$sudo docker inspect ubuntu /* Display low lever information about image */

Docker Basic Commands

- docker create ubuntu
- docker create --name test ubuntu
- docker create -i -t --name user1 ubuntu /bin/bash
- docker start user1
- docker attach user1
- docker stop user1
- $docker start -a -i user1 (it will start and attach)

Docker Basic Commands

- Demo : create_100.sh; delete_100.sh; create_10.sh; start-10.sh
- docker stats /* Monitor Running Container like top command */
- docker stats <container name>
- docker logs <running container id>
- docker logs -f <running container id> /* Real Time */
- docker info
- docker history <image name>
- docker search <image name>
- Remove all the containers: docker rm $(docker ps -a -q)
- Remove docker images: docker rmi <id>

Docker Basic Commands

copy file from host to docker viceversa:

- docker cp user.txt user1:<path>/user.txt (host to docker (user1))
- docker cp user1:user.txt /home/raju/user.txt (docker to host)

Run Container as background process:

- docker run -d -it --name user3 ubuntu /bin/bash
- docker attach user3 (if you want to get access)

Check Running Container information:

/var/lib/docker/containers /* List all the running containers */

Check log file for a specific container.

Docker Access through SSH

- create user account : adduser user1
- set password for user1
- create user1 as sudo user: usermod -aG sudo user1
- create user1 container
- Modify /home/user1/.profile file as:
- sudo docker start -a -i user1; exit
- ssh to <ip address> login as user1 and you will get user1 container shell.

Docker Web Server

- run user1 docker container
- apt-get update
- apt-get install -y apache2
- apt-get install -y apache2-utils
- apt-get install vim
- vim /var/www/html/index.html
- change line number 208:
- service apache2 start
- elinks <container ip address>

Docker Web Server

Execute webserver:

docker pull thangaraju/webserver_test

docker run -i -t -p 9999:80 thangaraju/webserver_test /bin/bash

service apache2 start

go to host system and execute in a browser:

<host ip address:9999>

Docker Image Creation

Docker image creation and move to docker hub

docker commit -a <Author Name> <container id> <image name>

docker tag <image id> <username/image name:latest>

$docker commit -a "Dr B Thangaraju" <container id> <container image name>

$docker tag <old name> <new name>

ex: docker tag jenkins_slave iiitb/jenkins_slave:latest

docker login (iiitb)

docker push <username/image name>

dockerhub.com - edit and verify

Docker Volume

1. **Create docker volume**

sudo docker volume create <volume_name>

ex: sudo docker volume create devtest

2. **List docker volume**

sudo docker volume list

3. **Inspecting docker volume**

ex: sudo docker volume inspect devtest

4. **Mounting docker volume**:

sudo docker run −−mount source=<name of volume>,destination=<path of a directory in container>,<readonly> <image_name>

ex: sudo docker run −it −−mount source=devtest,destination=/usr/src/app/ ubuntu /bin/bash

Docker Volume

After successful exection of the above command, you will get docker shell prompt

cd /usr/src/app /* store all the files, which you want to keep it in the volume */

ex: touch abc{1..100}.txt /* all 100 files will be stored in docker volume. */; exit.

5. **Now any container can connect to the volume and access the 100 files.**

ex: sudo docker run −it −−name=container1 −−mount source=devtest,destination=/app ubuntu

After successful execution of the above command, you will get docker shell prompt.

cd /app; $ls /*you can see all the 100 files. so any container can connect the volume and access the files*/

6. **Deleting docker volume:**

sudo docker rm <name of volume>

ex: sudo docker volume rm devtest

Docker for Lab

```
#!/bin/bash
echo -e -n "\t\tPlease enter how many systems you want: "; read no;
echo -e "\n\t\t 1. For C/C++/Python/Ruby/UNIX Systems Programming Course"
echo -e "\n\t\t 2. For JAVA Course "
echo -e "\n\t\t 3. For Debugging Techniques and Performance Tuning Courses"
echo -e "\n\t\t 4. For UNIX System Administration Course"
echo -e "\n\t\t 5. For UNIX L1 System Admin Hands-On Based Assessment"
echo -e -n "\n\n\t\t\t\t\tPlease Choose [1-5] :"; read choice
```

```
if [ $choice = 1 ]
 then
   image="developer"
 elif [ $choice = 2 ]
   then
     image="java"
   elif [ $choice = 3 ]
     then
       image="debugger"
     elif [ $choice = 4 ]
       then
        image="unixl1admin"
       elif [ $choice = 5 ]
        then
          image="unix_l1_assessment_base"
 else
    echo " ENTER valid choice"; exit
fi

 for ((i=1; i<=$no; i++))
  do
    useradd user$i
    echo "user$i@123" | passwd --stdin user$i
    docker create -i -t --name user$i $image /bin/bash
    echo "docker start -a -i user$i" >> /home/user$i/.bash_profile
    echo "exit" >> /home/user$i/.bash_profile
  done
```

```
REPOSITORY              TAG       IMAGE ID       CREATED        VIRTUAL SIZE
unixl1admin             latest    2bfea28d5dd0   3 weeks ago    260.6 MB
debugger                latest    bd068bff3086   3 weeks ago    474.4 MB
java                    latest    435cc2fcfb2c   3 weeks ago    571.2 MB
developer               latest    57f44d0451c3   3 weeks ago    341.1 MB
unix_l1_assessment_base latest    8f9caf92815b   3 weeks ago    325 MB
```

```
# Delete user accounts and containers.
#!/bin/bash
for ((i=1; i<=25; i++))
 do
   userdel user$i
   rm -rf /home/user$i
   var1=`docker ps -a | grep user$i | awk '{print $1}'`
   docker rm $var1
 done
```

Dockerfile

- vim Dockerfile

```
FROM ubuntu
RUN apt update
RUN apt install -y apache2
RUN apt install -y apache2-utils
RUN apt clean
EXPOSE 80 CMD ["apache2ctl", "-D", "FOREGROUND"]
```

- sudo docker build -t mywebserver:0.1 .
- http://ipaddress







THANK YOU